

LightRAG + LLM Wiki

混合知識系統完整建置手冊

證據檢索層 + 長期知識整理層 + Agent/MCP 操作層

版本：v1.2（整合討論修訂版） | 日期：2026-06-11 | 適用：Ubuntu / macOS / MiniMax + 本地 Embedding

層級	角色	建議工具
原始資料層	保存 PDF / Markdown / README / 程式碼 / 手冊，不直接被覆寫	Git + raw_sources/
證據檢索層	查原文、找引用、建立 entity / relation / vector / graph	LightRAG Server
知識整理層	把資料整理成可讀 Wiki，長期維護、互連、可被人與 Agent 閱讀	Markdown Wiki / LLM Wiki pattern
操作決策層	依問題決定先查 Wiki 或 LightRAG，必要時更新 Wiki	Agent / MCP / Claude Code / Codex / Hermes

本手冊的目標不是單純安裝兩個工具，而是建立一套可以長期累積專案知識的架構。LightRAG 負責「找證據」，LLM Wiki 負責「整理與維護知識」，Agent/MCP 負責「使用與決策」。

v1.1 修訂說明

本版在 v1.0 基礎上，將實測查證與討論結論補入對應章節，新增內容均以黃色標示框呈現，包括：

- **第 2 章**：混合架構下 raw_sources 與原版 LLM Wiki 模式的關係釐清。
- **第 5、6 章**：安裝與 .env 設定的三個實務陷阱（env.example 取得方式、`$USER` 變數展開、舊變數 ENTITY_TYPES、MiniMax 模型名稱確認）。
- **第 7 章**：raw_sources 與 LightRAG 上傳的差異、內容去重機制、INPUT_DIR 自動掃描模式、新資料端到端六步流程。
- **第 8 章**：Wiki 層整理邏輯的四個支柱與「總編輯」心態說明。
- **第 9、11、15、16 章**：編譯器引用結構驗證、lint 誤判、新增錯誤排除項目、參考來源驗證提醒。

v1.2 新增：第 17 章「進階缺口與補強建議」（藍色標示框），涵蓋知識圖譜刪除殘留、匯入成本估算、Wiki 知識回流決策、服務常駐化、卡住文件 SOP、還原演練、Git 衛生與檢索品質基準八個主題。

目錄

1. 架構總覽與設計原則
2. LightRAG 與 LLM Wiki 的角色分工 (含 v1.1 補充：raw_sources 關係釐清)
3. 推薦硬體與軟體配置
4. 專案資料夾結構 (含 v1.1 補充：inbox 待匯入區)
5. 安裝 LightRAG Server (含 v1.1 補充：env.example 取得方式)
6. MiniMax LLM + 本地 Embedding 設定 (含 v1.1 警告：三個設定陷阱)
7. 匯入文件與查詢驗證 (含 v1.1 補充：去重機制、自動掃描、六步流程)
8. 建立 Wiki 知識層 (含 v1.1 補充：整理邏輯四支柱)
9. LightRAG → Wiki 編譯器實作 (含 v1.1 補充：引用結構驗證)
10. 查詢流程與 Agent 路由
11. Wiki Lint 與一致性檢查 (含 v1.1 補充：孤兒頁誤判)
12. MCP / Agent 整合
13. PostgreSQL / Rerank / Multimodal 升級路線
14. 維運、安全與備份
15. 常見錯誤排除 (v1.1 新增三項)
16. 參考來源 (含 v1.1 驗證提醒)
17. 進階缺口與補強建議 (v1.2 新增)

附錄 A：最小可行驗收清單 | 附錄 B：建議落地順序

1. 架構總覽與設計原則

這套混合架構的核心原則是：**不要讓 RAG 每次都從零開始回答，也不要讓 Wiki 失去原文證據。**

整體資料流



LightRAG 官方文件建議整合專案優先使用 REST API Server，而不是直接把 SDK 嵌進專案；Server 同時提供 WebUI、API、Ollama-compatible interface，適合作為底層檢索服務。

LLM Wiki 的核心思想則是 Raw Sources → Wiki → Schema：原始資料保持不可變，Wiki 由 LLM 編譯與維護，Schema/AGENTS.md/CLAUDE.md 規範頁面格式、命名、引用與 lint 規則。

1.1 為什麼要結合，而不是二選一？

只用 LightRAG 的問題	只用 LLM Wiki 的問題	混合架構的解法
每次查詢仍偏「即時拼答案」，長期知識沉澱較弱	摘要可能漏掉細節，且需要引用原始來源	LightRAG 找證據，Wiki 保存整理後的知識
人類不容易直接閱讀向量索引與圖譜	Wiki 頁面可能互相矛盾或過期	Wiki 需要定期 Lint，必要時回查 LightRAG
Agent 需要懂 query mode、references、chunks	Agent 容易只看整理內容，不看原始證據	Agent 先讀 Wiki，不足時查 LightRAG，並更新 Wiki

1.2 最終系統應具備的能力

- 可以把 PDF、Markdown、README、技術文件、程式碼說明匯入 LightRAG。
- 可以透過 LightRAG API 以 mix / hybrid / local / global 模式查詢，並取回 references 與 chunk content。
- 可以把查詢結果編譯成 Wiki 頁面，頁面具有 YAML frontmatter、[[wikilink]]、來源引用與更新紀錄。
- 可以讓 Claude Code、Codex、Hermes 或其他 Agent 使用 Wiki 作為第一入口，LightRAG 作為證據回查工具。
- 可以定期執行 Wiki lint，找出孤兒頁、無來源頁、矛盾敘述、過期頁與待人工審核項目。

2. LightRAG 與 LLM Wiki 的角色分工

模組	負責事情	不該負責的事情
LightRAG	索引原始資料、抽取實體關係、向量檢索、圖檢索、回傳 references	長期寫作、主題頁規劃、人工可讀的知識庫維護
Wiki Compiler	把 LightRAG 證據整理為 Wiki 頁、更新 index/log、產生 wikilink	取代原始資料、憑空補知識、無來源改寫
Wiki	給人與 Agent 閱讀、作為長期知識沉澱	保證所有細節完整無誤；細節仍要回查 LightRAG
Agent/MCP	判斷路由、查詢、寫回、審核、觸發 lint	在沒有來源的情況下新增事實

建議規則：凡是「事實」都要有來源；凡是「推論」都要標註推論；凡是「決策」都要留下理由與日期。

v1.1 補充

2.1 混合架構下 raw_sources 與原版 LLM Wiki 模式的關係

LLM Wiki 模式本身（Raw Sources → Wiki → Schema）就內建原始資料層的設計，這正是兩套系統能夠混合的接合點。**混合架構沒有取消 raw_sources 層**——它原封不動保留，仍是不可變的真相來源；改變的是「Wiki 是怎麼讀到原始資料的」。

原版 LLM Wiki：raw_sources/ → Agent 親自翻檔案讀全文 → 整理成 Wiki
混合架構：raw_sources/ → LightRAG（向量+圖譜索引）→ 回傳相關段落+引用 → Wiki

原版模式在資料量小時可行，但有三個天花板：(1) Context window 限制，長文件塞不進一次對話；(2) 文件多時 Agent 不知道答案在哪份檔的哪一章，要嘛全翻要嘛猜錯；(3) 跨文件關聯弱。

	原版 LLM Wiki	混合架構
raw_sources 角色	不可變真相來源	完全相同，不變
Wiki 結構與 schema	index / log / wikilink / lint	完全相同，不變
找資料的方式	Agent 逐檔翻閱	向 LightRAG 查詢，語意檢索直接命中相關 chunk
引用的形式	檔案路徑	檔案路徑 + chunk 內容摘錄（更精確）
跨文件問題	靠 Agent 記憶力	靠知識圖譜的 entity/relation
適合規模	幾份到十幾份文件	數十到數百份文件

LightRAG 取代的不是 raw_sources，而是「Agent 翻檔案的眼睛和手」。原版模式裡 Agent 是親自跑書庫找書的圖書館員；混合模式裡圖書館多了一套索引卡片系統，館員先查卡片直接定位到第幾排書架第幾頁。

直接讀檔的權利仍保留：raw_sources 就在硬碟上，當 LightRAG 回傳的 chunk 不完整需要看前後文、要逐字核對數值或程式碼、或文件很短直接讀更快時，Agent 仍可直接讀檔。LightRAG 是預設的檢索入口（處理「在哪裡」），直接讀檔是精讀的後備手段（處理「逐字確認」），兩條路徑下 raw_sources 都是同一份、都不可變。

3. 推薦硬體與軟體配置

3.1 你的 Mac mini M2 Pro 16GB 配置建議

元件	建議	理由
LLM	MiniMax API	避免本機 16GB RAM 承擔大型 LLM 推理；把本機資源留給 embedding、索引與檔案處理
Embedding	Ollama + bge-m3 或其他 1024 維 multilingual embedding	LightRAG 文件建議 embedding 要先選定；更換後通常需要重建索引
Storage	第一版用本地 file storage，第二版升 PostgreSQL	第一版容易啟動；確定資料量與流程後再升級資料庫
Wiki	Markdown + Git + Obsidian	可版本控管、可視化連結、可人工審核
Agent	Claude Code / Codex / Hermes 任一種	只要能讀寫檔案、呼叫 API，就能套用本架構

3.2 Ubuntu 建議配置

- Python 3.11 或 3.12。
- uv：負責 Python 環境與工具安裝。
- Ollama：本地 embedding 服務。
- Git：管理 raw_sources 與 wiki 版本。
- 可選 Docker / Docker Compose：後續 PostgreSQL、OpenWebUI、Docling/MinerU 或其他服務。

4. 專案資料夾結構

```
~/hybrid-knowledge-lab/
├── raw_sources/          # 原始資料，不要讓 LLM 修改
│   ├── lightrag/
│   ├── project_docs/
│   └── papers/
├── lightrag_workspace/  # LightRAG 儲存資料，可備份但不要手改
├── wiki/                # LLM Wiki 層，可用 Obsidian 開啟
│   ├── index.md
│   ├── log.md
│   ├── entities/
│   ├── concepts/
│   ├── sources/
│   ├── synthesis/
│   ├── comparisons/
│   └── queries/
├── schema/
│   ├── AGENTS.md        # 給 Agent 的 Wiki 維護規則
│   ├── wiki_page_template.md
│   └── lint_rules.yml
├── scripts/
│   ├── ingest_sources.py
│   ├── query_lightrag.py
│   ├── compile_wiki_page.py
│   ├── wiki_lint.py
│   └── route_query.py
├── outputs/
├── .env
└── README.md
```

建立資料夾

```
mkdir -p ~/hybrid-knowledge-lab/{raw_sources/lightrag,raw_sources/project_docs,raw_sources/papers,lightrag_workspace,wiki/{entities,concepts,sources,synthesis,comparisons,queries},schema,scripts,outputs}
cd ~/hybrid-knowledge-lab
git init
cat > .gitignore <<'EOF'
.env
lightrag_workspace/
__pycache__/.
.venv/
*.log
EOF
```

v1.1 補充：可選的 inbox/ 待匯入區

若後續採用第 7.6 節的自動掃描模式，建議在 raw_sources 下多建一個 `inbox/` 待匯入區，把 INPUT_DIR 指向它，讓「丟進 inbox = 我確定要餵給 LightRAG」的意圖明確，避免整理舊檔或放入未完成草稿時被意外吃進索引。檔案處理完可留在原地直接當歸檔的一部分（移動檔案會使 LightRAG 記錄的 file_path 變成舊路徑）。第一版手動上傳階段不需要此資料夾。

5. 安裝 LightRAG Server

以下以 Ubuntu 為主要範例；macOS 大致相同，只是系統套件改用 Homebrew。

5.1 安裝基本工具

```
sudo apt update
sudo apt install -y git curl build-essential python3 python3-venv python3-pip
curl -LsSf https://astral.sh/uv/install.sh | sh
source ~/.bashrc
uv --version
```

5.2 安裝 Ollama 與 bge-m3

```
curl -fsSL https://ollama.com/install.sh | sh
ollama serve
# 另開一個 terminal
ollama pull bge-m3
ollama list
```

注意：embedding model 一旦用來建立 LightRAG 索引，後續不要輕易更換。若要更換 embedding，通常要清掉向量資料並重建索引。

5.3 安裝 LightRAG Server

```
# 建議方式：用 uv tool 安裝 API/WebUI 版
uv tool install "lightrag-hku[api]"
# 確認指令可用
lightrag-server --help
```

若你想保留完整原始碼、方便查看 env.example 或修改 WebUI，可改用 source 安裝：

```
cd ~/hybrid-knowledge-lab
git clone https://github.com/HKUDS/LightRAG.git vendor/LightRAG
cd vendor/LightRAG
uv sync --extra test --extra offline
source .venv/bin/activate
cp env.example ~/hybrid-knowledge-lab/.env
```

v1.1 補充：env.example 的取得方式與 .env 讀取位置

用 `uv tool install` 安裝時，LightRAG 是全域 CLI 工具，**當前目錄不會有 env.example**——必須手動建立 .env，或從 GitHub 倉庫根目錄下載 env.example 作為範本。另外 `lightrag-server` 啟動時是從**目前工作目錄讀取 .env**，所以務必先 `cd ~/hybrid-knowledge-lab` 再啟動。本手冊第 6 章的設定變數名稱仍應與你安裝版本的最新 env.example 逐一比對。

6. MiniMax LLM + 本地 Embedding 設定

LightRAG 支援 openai / OpenAI-compatible provider。MiniMax 若使用 OpenAI-compatible Chat Completions，概念上就是把 LLM_BINDING 設為 openai，host 改成 MiniMax 的 base URL，API key 改成你的 MiniMax key。

```
cd ~/hybrid-knowledge-lab
cat > .env <<'EOF'
# Server
HOST=0.0.0.0
PORT=9621
WORKING_DIR=/home/$USER/hybrid-knowledge-lab/lightrag_workspace

# Language and extraction
SUMMARY_LANGUAGE=Chinese
ENTITY_EXTRACTION_USE_JSON=true
MAX_ASYNC_LLM=4
MAX_PARALLEL_INSERT=1
TIMEOUT=180

# Parser: 第一版先用內建與 legacy，不先導入外部 OCR/Docling/MinerU
LIGHTRAG_PARSER=:native-tep,:legacy-R
VLM_PROCESS_ENABLE=false

# LLM: MiniMax OpenAI-compatible。請依 MiniMax 平台最新文件確認 host/model 名稱。
LLM_BINDING=openai
LLM_BINDING_HOST=https://api.minimax.io/v1
LLM_BINDING_API_KEY=YOUR_MINIMAX_API_KEY
LLM_MODEL=minimax-m3
KEYWORD_LLM_MODEL=minimax-m3
QUERY_LLM_MODEL=minimax-m3

# Embedding: Ollama local bge-m3
EMBEDDING_BINDING=ollama
EMBEDDING_BINDING_HOST=http://localhost:11434
EMBEDDING_MODEL=bge-m3:latest
EMBEDDING_DIM=1024
EMBEDDING_FUNC_MAX_ASYNC=4
EMBEDDING_BATCH_NUM=16

# Query references/debug
RERANK_BINDING=null
RERANK_BY_DEFAULT=false

# Security - local lab can be simple; LAN/remote must enable API key + account auth
# LIGHTRAG_API_KEY=replace-with-long-random-key
# WHITELIST_PATHS=/health
EOF
```

若你使用的是中國站 MiniMax 帳號，base URL 可能不是 api.minimax.io，而是 minimaxi.com 相關網域；請以你平台控制台顯示的 endpoint 為準。

v1.1 警告：三個 .env 設定陷阱

- \$USER 不會展開**：上面用 <<'EOF'（單引號 heredoc）建檔，WORKING_DIR=/home/\$USER/... 會以字面 \$USER 寫入檔案。雖然 6.1 節用 set -a; source .env 可透過 shell 補救，但若讓 lightrag-server 直接讀 .env 就會出錯。**建議直接寫死絕對路徑**，例如 WORKING_DIR=/home/你的帳號/hybrid-knowledge-lab/lightrag_workspace。
- 舊變數 ENTITY_TYPES 會導致啟動失敗**：官方文件指出此變數已被 prompt profiles 取代，若 .env 殘留 ENTITY_TYPES，server 會在啟動時直接失敗（fail fast）。沿用舊範本時務必移除。
- minimax-m3 模型名稱需自行確認**：實際可用的 model ID 請以 MiniMax 控制台為準（截至查證時公開旗艦為 M2 系列）。另外 ENTITY_EXTRACTION_USE_JSON、KEYWORD_LLM_MODEL、QUERY_LLM_MODEL 等變數名稱在版本間可能變動，啟動前請與最新 env.example 比對。

6.1 啟動 LightRAG

```
cd ~/hybrid-knowledge-lab
set -a
source .env
set +a
lightrag-server
```

啟動後打開：

- WebUI / API docs: <http://localhost:9621/docs>
- Health check: <http://localhost:9621/health>

7. 匯入文件與查詢驗證

7.1 放入原始資料

```
cd ~/hybrid-knowledge-lab
cp ~/Downloads/*.md raw_sources/project_docs/ 2>/dev/null || true
cp ~/Downloads/*.pdf raw_sources/papers/ 2>/dev/null || true
```

7.2 用 WebUI 匯入

1. 打開 `http://localhost:9621`。
2. 進入 Documents / Upload。
3. 上傳 `raw_sources` 中的檔案。
4. 等 Track ID 狀態變成 `processed`。
5. 用 Query 頁面測試 `mix / hybrid` 模式。

7.3 用 API 匯入與查詢

```
# 查詢健康狀態
curl http://localhost:9621/health

# 查詢範例：要求 references 與 chunk content，方便後續編譯 Wiki
curl -X POST http://localhost:9621/query \
-H "Content-Type: application/json" \
-d '{
  "query": "請整理這批文件中的系統架構與主要模組",
  "mode": "mix",
  "include_references": true,
  "include_chunk_content": true
}' | jq .
```

LightRAG 的 `/query` 端點支援 `include_references` 與 `include_chunk_content`，這對 Wiki 編譯器非常重要，因為 Wiki 不能只保存答案，還要保存引用來源。

v1.1 補充

7.4 `raw_sources` 歸檔與 LightRAG 上傳的差異

步驟「放入 `raw_sources`」與「上傳到 LightRAG」雖然動到同一份檔案，但是兩個完全獨立的儲存位置：

	放入 <code>raw_sources/</code>	上傳到 LightRAG
本質	把檔案歸檔到你自己的資料夾	把檔案餵給 LightRAG 去消化
檔案去哪	留在 <code>raw_sources/</code> ，原封不動	被複製進 LightRAG 管理範圍，切碎成 chunk、向量、實體，存進 <code>lightrag_workspace/</code>
角色	母本、檔案櫃、最終真相來源	加工品、索引、可隨時砍掉重建
可否改/刪	永遠保留，用 Git 管理	可整個刪掉重建（例如換 embedding 時）

比喻：`raw_sources/` 是書櫃——書買回來先上架，永遠在那裡；上傳到 LightRAG 是把書拿給助理讀——助理讀完做了一份卡片索引。索引弄丟了沒關係，書還在書櫃上，再讓他讀一次就好。

WebUI 上傳不會自動去讀 `raw_sources/`，上傳時是把檔案複製一份進 LightRAG 自己的目錄。所以兩步缺一不可：只歸檔不上傳，LightRAG 查不到它；只上傳不歸檔，換 embedding 或 workspace 損毀要重建索引時會找不到原始檔。一句話記法：要分析的檔案永遠先放 `raw_sources/`，再從那裡上傳給 LightRAG。Downloads 裡的原始下載檔可以刪，`raw_sources` 裡的不能刪。

7.5 重複處理與內容去重機制

`raw_sources` 越放越多，不會導致舊檔被重複處理，有兩層保護：

1. 上傳行為由你控制：WebUI 手動上傳是「你選哪份，它處理哪份」，LightRAG 不會主動掃描 `raw_sources`。
2. 內容 hash 去重：LightRAG 用文件內容的 MD5 hash 產生文件 ID（`doc-xxx`），匯入時先查 `doc_status` 資料庫，把已存在的文件 ID

過濾掉，只處理真正的新文件。即使不小心重複上傳同一份檔，只要內容一字不差，就會被認出並跳過昂貴的 LLM 抽取流程。

兩個邊界情況要留意：

- 「同一份文件的修改版」會被當成新文件。去重看的是內容 hash，改了一個字再上傳就是全新文件，而且舊版索引還留在裡面，查詢時新舊內容可能混在一起。正確做法：更新文件時，先在 WebUI 的 Documents 頁刪除舊版，再上傳新版。
- WebUI 的重複上傳提示在部分版本不夠明顯，曾有使用者回報重複上傳同一檔案時介面未跳出警告。不要依賴介面提醒，上傳前自行到 Documents 列表確認該檔是否已是 processed。

建議讓 wiki/log.md 幫你記帳，每次匯入新檔 append 一筆：

```
## [2026-06-11] ingest | XX晶片規格書 v2.pdf
- 路徑: raw_sources/papers/xx_spec_v2.pdf
- LightRAG 狀態: processed
- 備註: 取代 v1，已從 LightRAG 刪除 v1 索引
```

7.6 進階：INPUT_DIR + /documents/scan 自動掃描模式

除了手動上傳，LightRAG Server 還支援「指定一個資料夾，讓它自己去掃」：在 .env 設定 INPUT_DIR（預設 ./inputs）指向收件匣，呼叫 /documents/scan API（或在 WebUI 按 Scan 按鈕）時，伺服器會遞迴掃描該資料夾、依 LIGHTRAG_PARSER 設定處理檔案，並搭配內容 hash 去重，只把新檔案排入處理佇列。

```
# .env 加入（或指向 raw_sources/inbox，見第 4 章補充）
INPUT_DIR=/home/你的帳號/hybrid-knowledge-lab/raw_sources

# 之後的流程簡化為：
cp ~/Downloads/新資料.pdf ~/hybrid-knowledge-lab/raw_sources/papers/
curl -X POST http://localhost:9621/documents/scan
# 到 WebUI Documents 頁確認新檔變成 processed
```

第一版先不要用的理由：(1) 手動上傳看得到每份檔的處理過程，出錯馬上知道是哪份；(2) 自動掃描是資料夾裡全部照單全收，不小心丟入草稿或超大 PDF 也會消耗 MiniMax token；(3) 若 raw_sources 已堆了幾十份檔，第一次 scan 會全部排隊處理，對 16GB 機器與 API 額度都是壓力。等流程跑順、確認去重如預期、.env 與 parser 設定穩定後再切換。

7.7 新資料端到端處理流程總覽

一份新資料從進入系統到沉澱為知識，完整走六步（跨第 7、9、11、14 章）：

步驟	動作	說明
1. 歸檔	放入 raw_sources/ 對應子資料夾	留底，從此不被任何 LLM 改動（7.1）
2. 匯入	WebUI 上傳（或 scan），等 processed	耗時最久的一步，MiniMax 被多次呼叫做實體抽取（7.2）
3. 索引	LightRAG 自動建 chunk/向量/實體/圖譜	本地 bge-m3 算向量，結果存 lightrag_workspace/（自動）
4. 編譯	針對新文件想 3-5 個問題，跑 compile_wiki_page.py	答案+引用寫成 wiki/queries/*.md，狀態 needs_review（第 9 章）
5. 審核	跑 wiki_lint.py，人工審核 needs_review 頁	確認被引用支撐、補 wikilink、必要時整併，改 verified（第 11 章）
6. 備份	git commit + 打包 workspace	第 14 章備份策略

之後查詢走另一條路：先搜 wiki/ → 命中直接回答 → 沒命中或需原文細節才查 LightRAG → 新結論寫回 Wiki。資料處理得越勤，日常查詢就越快越省 token。

8. 建立 Wiki 知識層

Wiki 層不是把 LightRAG 的答案直接貼上，而是把「已查證的資訊」整理成長期頁面。建議用 Markdown，原因是：可讀、可 Git 版本控管、可被 Obsidian 開啟、可被 Agent 直接讀寫。

```
cd ~/hybrid-knowledge-lab
cat > wiki/index.md <<'EOF'
---
title: Hybrid Knowledge Wiki Index
type: index
updated: 2026-06-11
---
# Hybrid Knowledge Wiki Index

## 核心入口
- [[concepts/system_architecture]]
- [[concepts/lightrag_layer]]
- [[concepts/wiki_layer]]
- [[concepts/agent_routing]]

## 最近更新
請查看 [[log]]。
EOF

cat > wiki/log.md <<'EOF'
---
title: Operation Log
type: log
---
# Operation Log

## [2026-06-11] init | 建立 Hybrid Knowledge Wiki
- 建立 index.md 與基本資料夾。
EOF
```

8.1 Wiki 頁面標準格式

```
---
title: 頁面標題
type: concept | entity | source | synthesis | comparison | query
status: draft | verified | needs_review
updated: YYYY-MM-DD
source_count: 0
related:
  - [[concepts/example]]
---
# 頁面標題

## 摘要
用 3-5 句話說明這頁的核心內容。

## 核心知識
- 事實 A。[^src1]
- 事實 B。[^src2]

## 與其他頁面的關係
- 參見 [[concepts/xxx]]。

## 開放問題
- [ ] 哪些地方需要回查原始資料？

## Sources
[^src1]: raw_sources/... 或 LightRAG reference id / file path / chunk 摘要
[^src2]: raw_sources/... 或 LightRAG reference id / file path / chunk 摘要
```

8.2 AGENTS.md：讓 Agent 嚴格維護 Wiki

schema/AGENTS.md

```
# Hybrid Knowledge Wiki Maintainer Rules

You maintain the wiki directory as a long-term knowledge base.

Rules:
1. Never overwrite raw_sources.
2. Every factual claim added to wiki pages must include a source note.
3. Prefer updating existing pages over creating near-duplicate pages.
4. Use [[wikilink]] for related entities, concepts, projects, tools, and decisions.
5. If evidence is insufficient, add the statement under "Open Questions" instead of writing it as fact.
6. Append every ingest/query/lint/update action to wiki/log.md using this format:
  ## [YYYY-MM-DD] action | title
7. When new evidence contradicts old content, do not silently replace it. Add a "Conflict / Revision" section.
8. Use Traditional Chinese for human-facing explanations unless the source title/code/API name is English.
```

v1.1 補充

8.3 Wiki 層整理邏輯的四個支柱

可以把 Wiki 層理解成「一個有嚴格規矩的 Obsidian 筆記庫，但寫筆記的人是 LLM，你是總編輯」。它的整理邏輯有四個支柱：

支柱一：用資料夾區分頁面類型——每種資料夾對應一種知識形態：

資料夾	放什麼	例子
entities/	具體的東西：一個工具、一顆晶片、一個 API	entities/bge_m3.md
concepts/	抽象概念與架構	concepts/agent_routing.md
sources/	對單一原始文件的描述頁	sources/lightrag_readme.md
synthesis/	跨多份文件綜合出的結論	「三種 RAG 方案的取捨」
comparisons/	明確的 A vs B 比較	「PostgreSQL vs 本地儲存」
queries/	編譯器產出的原始查詢頁（半成品入口暫存區）	compile_wiki_page.py 的輸出

實務上 queries/ 是入口暫存區：編譯器先丟到這裡，審核時你決定這頁該獨立成 concept、併入某個 entity 頁、還是留著就好。

支柱二：用 YAML frontmatter 管理每頁的生命週期——status: draft → needs_review → verified 是信任機制的核心：Agent 回答問題時，verified 頁可直接引用，needs_review 頁要提醒「尚未人工確認」。updated 是判斷過期的依據，source_count 顯示有幾個引用支撐，related 記錄關聯頁面。

支柱三：事實必附來源，防止 LLM 幻覺——每條事實後面掛腳註 [^srcN]，指向 LightRAG 引用（檔案路徑+chunk 摘錄）。沒有來源的陳述只能放「開放問題」區，不能寫成事實。wiki_lint.py 就是機械化巡邏這條規則——抓出標了 verified 卻沒有任何引用的頁面。

支柱四：三個機制讓知識「活著」——(1) index.md 是唯一入口頁，Agent 永遠從這裡開始讀，避免迷路或重複建頁；(2) log.md 是 append-only 操作日誌，讓你隨時能回答「這個結論是什麼時候、根據什麼加進來的」；(3) [[wikilink]] 讓頁面互連，在 Obsidian 可視化成知識圖，lint 找出沒人連到的孤兒頁提醒整併。

關鍵心態：Wiki 不是 LightRAG 答案的貼上區，而是已查證知識的長期沉澱。LightRAG 像研究助理，每次幫你翻原文找證據；Wiki 像你親自維護的教科書——助理交上來的報告（queries/ 頁）要經過審核、改寫、歸檔，才能成為教科書的正式章節（verified 的 concept/entity 頁）。AGENTS.md 就是寫給「LLM 編輯」的編輯部守則。

9. LightRAG → Wiki 編譯器實作

這一層是混合架構的核心：它會向 LightRAG 查詢，要求 references 與 chunk content，然後把結果寫入 Wiki。第一版先用簡單 Python 腳本，後續再升級成 MCP tool 或背景任務。

建立腳本環境

```
cd ~/hybrid-knowledge-lab
uv venv
source .venv/bin/activate
uv pip install requests pyyaml python-frontmatter rich
```

```
# scripts/query_lightrag.py
import os, json, requests

BASE_URL = os.getenv("LIGHTRAG_URL", "http://localhost:9621")
API_KEY = os.getenv("LIGHTRAG_API_KEY", "")

def query_lightrag(question: str, mode: str = "mix") -> dict:
    headers = {"Content-Type": "application/json"}
    if API_KEY:
        headers["X-API-Key"] = API_KEY
    payload = {
        "query": question,
        "mode": mode,
        "include_references": True,
        "include_chunk_content": True,
        "enable_rerank": False,
    }
    r = requests.post(f"{BASE_URL}/query", headers=headers, json=payload, timeout=240)
    r.raise_for_status()
    return r.json()

if __name__ == "__main__":
    import sys
    q = " ".join(sys.argv[1:]) or "請整理主要架構"
    print(json.dumps(query_lightrag(q), ensure_ascii=False, indent=2))
```

```
# scripts/compile_wiki_page.py
import re, sys, json
from pathlib import Path
from datetime import date
from query_lightrag import query_lightrag

ROOT = Path(__file__).resolve().parents[1]
WIKI = ROOT / "wiki"

def slugify(text: str) -> str:
    text = text.lower().strip()
    text = re.sub(r"^[a-z0-9\u4e00-\u9fff]++", "_", text)
    return text.strip("_")[:80] or "untitled"

def refs_to_markdown(refs):
    lines = []
    for i, ref in enumerate(refs or [], 1):
        fp = ref.get("file_path", "unknown")
        chunks = ref.get("content", [])
        preview = " ".join(chunks[:300]).replace("\n", " ") if chunks else ""
        lines.append(f"^[src{i}]: {fp}。摘錄：{preview}")
    return "\n".join(lines)

def compile_page(title: str, question: str, folder: str = "queries"):
    result = query_lightrag(question, mode="mix")
    answer = result.get("response", "")
    refs = result.get("references", [])
    slug = slugify(title)
    out_dir = WIKI / folder
    out_dir.mkdir(parents=True, exist_ok=True)
    out = out_dir / f"{slug}.md"
    md = f"""---
title: {title}
type: {folder.rstrip('s')}
status: needs_review
updated: {date.today().isoformat()}
source_count: {len(refs)}
related: []
---
# {title}

## 查詢問題
{question}

## LightRAG 整理結果
{answer}

## 待人工審核
- [ ] 檢查答案是否完全被 Sources 支撐。
- [ ] 補上必要的 [[wikiLink]]。
- [ ] 若此頁應整併到既有 concept/entity 頁，請移動內容。

## Sources
{refs_to_markdown(refs)}
"""
    out.write_text(md, encoding="utf-8")
    with (WIKI / "log.md").open("a", encoding="utf-8") as f:
        f.write(f"\n## [{date.today().isoformat()}] compile | {title}\n- Question: {question}\n- Output: {out.relative_to(WIKI)}\n")
    print(out)

if __name__ == "__main__":
    title = sys.argv[1]
    question = " ".join(sys.argv[2:])
    compile_page(title, question)
```

```
# 使用方式
cd ~/hybrid-knowledge-lab
source .venv/bin/activate
python scripts/compile_wiki_page.py "LightRAG 系統架構" "請整理 LightRAG 的系統架構、儲存層與查詢模式"
```

v1.1 補充：先實測 /query 回傳的 JSON 結構

compile_wiki_page.py 的 refs_to_markdown 假設每個 reference 含 `file_path` 與 `content` (list) 欄位，但實際回傳的 schema 可能因 LightRAG 版本不同（例如欄位名為 `reference_id`、`content` 為字串而非 list）。正式使用前先用 7.3 節的 `curl` 指令跑一次真實查詢，觀察實際 JSON 結構，再調解析邏輯，避免編譯出來的 Sources 區全部是 unknown。

10. 查詢流程與 Agent 路由

Agent 不應每次都直接呼叫 LightRAG。最佳流程是先查 Wiki，因為 Wiki 是整理過的長期知識；若 Wiki 沒答案、答案過期、或需要原文證據，再查 LightRAG。

使用者問題

↓

判斷問題類型

- └─ 已知專案概念 / 操作流程 / 既有決策 → 先查 wiki/
- └─ 細節證據 / 原始文件 / 引用來源 → 查 LightRAG
- └─ Wiki 與 LightRAG 衝突 → 標記 needs_review
- └─ 新的高價值結論 → 寫回 wiki/ 並 append log.md

```
# scripts/route_query.py
from pathlib import Path
from query_lightrag import query_lightrag

ROOT = Path(__file__).resolve().parents[1]
WIKI = ROOT / "wiki"

def search_wiki(keyword: str):
    hits = []
    for p in WIKI.rglob("*.md"):
        txt = p.read_text(encoding="utf-8", errors="ignore")
        if keyword.lower() in txt.lower():
            hits.append((p, txt[:800]))
    return hits[:5]

def answer(question: str):
    # 第一版簡化：用問題前 8 個中文字/詞當 wiki 搜尋 key；正式版可換 BM25/vector search
    key = question[:8]
    hits = search_wiki(key)
    if hits:
        return {"route": "wiki", "hits": [str(h[0]) for h in hits]}
    else:
        return {"route": "lightrag", "result": query_lightrag(question)}

if __name__ == "__main__":
    import sys, json
    print(json.dumps(answer(" ".join(sys.argv[1:])), ensure_ascii=False, indent=2))
```


11. Wiki Lint 與一致性檢查

LLM Wiki 最大風險是整理後內容失真、互相矛盾、沒有來源或過期。因此每次批量更新後都要做 lint。

```
# scripts/wiki_lint.py
from pathlib import Path
import re

ROOT = Path(__file__).resolve().parents[1]
WIKI = ROOT / "wiki"

problems = []
all_pages = list(WIKI.rglob("*.md"))
all_text = "\n".join(p.read_text(encoding="utf-8", errors="ignore") for p in all_pages)

for p in all_pages:
    txt = p.read_text(encoding="utf-8", errors="ignore")
    rel = p.relative_to(WIKI)
    if p.name not in ["index.md", "log.md"]:
        if "## Sources" not in txt:
            problems.append((rel, "missing Sources section"))
        if "status: verified" in txt and "[^src" not in txt:
            problems.append((rel, "verified page without citations"))
        title = re.search(r"^title:\s*(.+)$", txt, re.M)
        if not title:
            problems.append((rel, "missing YAML title"))
        # 找孤兒頁：若沒有任何 [[path_stem]] 被引用，先列為提醒，不一定是錯
        stem = p.with_suffix('').relative_to(WIKI).as_posix()
        if p.name not in ["index.md", "log.md"] and f"{{{stem}}}" not in all_text:
            problems.append((rel, "possible orphan page"))

for rel, msg in problems:
    print(f"{rel}: {msg}")
print(f"Total problems: {len(problems)}")
```

```
cd ~/hybrid-knowledge-lab
source .venv/bin/activate
python scripts/wiki_lint.py
```

v1.1 補充：孤兒頁偵測的誤判

上面的孤兒頁偵測只匹配完整路徑形式的 `[[concepts/xxx]]`；若你或 Agent 用 Obsidian 慣用的短連結 `[[xxx]]`，該頁會被誤判為孤兒頁產生噪音。解法擇一：(1) 在 AGENTS.md 規定一律使用完整路徑 wikilink（建議，最簡單）；(2) 修改 lint 同時匹配 `[[{stem}]]` 與 `[[{頁面檔名}]]` 兩種形式。

12. MCP / Agent 整合

第一版不需要一開始就寫完整 MCP Server。你可以先把 scripts/ 當作 Agent 可呼叫的命令工具；穩定後再包成 MCP。

12.1 Claude Code / Codex 使用規則

給 Agent 的提示詞範本：

```
你正在維護 ~/hybrid-knowledge-lab。
請先閱讀 schema/AGENTS.md。
回答任何專案知識問題時，請遵守：
1. 先查 wiki/index.md 與相關 wiki 頁面。
2. 若 Wiki 資訊不足，執行 python scripts/query_Lightrag.py "問題" 回查原文證據。
3. 若得到高價值新結論，更新或新增 wiki 頁面。
4. 每次更新 wiki 都要 append wiki/log.md。
5. 不要修改 raw_sources。
6. 請用繁體中文回覆我。
```

12.2 MCP 化的方向

MCP Tool	內部動作	用途
wiki_search(query)	搜尋 wiki/*.md	優先查長期知識
wiki_read(path)	讀取指定 Wiki 頁	讓 Agent 精讀
lightrag_query(query, mode)	呼叫 /query include_references	需要證據時回查
wiki_write(path, content)	寫入 Wiki 頁	更新知識
wiki_lint()	執行 lint 腳本	更新後檢查品質

若你直接使用 nashsu/llm_wiki 桌面版，它已朝本地 HTTP API、MCP Server、Agent skill 方向整合；你也可以把它當成 Wiki UI/Agent 入口，LightRAG 仍作為證據回查服務。

13. PostgreSQL / Rerank / Multimodal 升級路線

13.1 第一版：本地 file storage

- 最少依賴、最快啟動。
- 適合驗證流程與少量文件。
- 缺點是多人協作、資料一致性與擴充性較弱。

13.2 第二版：PostgreSQL

LightRAG 官方文件指出，正式環境可選 PostgreSQL / MongoDB / OpenSearch 作為四種 storage 的統一後端，或使用專門向量庫與圖資料庫。PostgreSQL 是你後續最實用的升級方向。

```
# docker-compose.postgres.yml 範例
services:
  postgres:
    image: pgvector/pgvector:pg16
    container_name: hybrid_lightrag_postgres
    environment:
      POSTGRES_DB: lightrag
      POSTGRES_USER: lightrag
      POSTGRES_PASSWORD: change-me
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

實際 LightRAG storage 環境變數名稱請以當前 env.example 為準；升級前務必先備份 lightrag_workspace 與 wiki。

13.3 第三版：Rerank

Rerank 適合查詢品質要求高、候選 chunk 很多的情境。LightRAG 文件指出 rerank 可提升查詢品質，但通常會增加 1-2 秒延遲；建議先把基礎檢索跑穩，再導入本地 reranker。

```
# 概念設定，實際 provider 依 env.example 支援為準
RERANK_BINDING=ollama_or_local_reranker
RERANK_BY_DEFAULT=true
MAX_ASYNC_RERANK=2
RERANK_TIMEOUT=30
```

13.4 第四版：Multimodal 文件處理

若你的文件包含大量 PDF 圖表、表格、公式，後續可導入 MinerU 或 Docling。第一版不建議先開，因為部署複雜度會明顯提高。

```
# MinerU / VLM 路線概念
LIGHTRAG_PARSER=*:native-iteP,*:mineru-iteP,*:Legacy-R
VLM_PROCESS_ENABLE=true
VLM_LLM_MODEL=your-vlm-model
MINERU_API_MODE=official_or_local
```

14. 維運、安全與備份

14.1 備份策略

```
cd ~/hybrid-knowledge-lab
# Wiki 用 git 管理
git add wiki schema scripts README.md
git commit -m "update wiki and hybrid scripts"
# LightRAG workspace 打包備份
tar -czf outputs/lightrag_workspace_$(date +%Y%m%d).tar.gz lightrag_workspace
```

14.2 安全策略

- 不要把 .env commit 到 Git。
- 若 LightRAG 開到 LAN 或遠端，必須設定 LIGHTRAG_API_KEY 與 WebUI 帳號。
- 不要讓 Agent 任意修改 raw_sources。
- 所有 Wiki 更新都應留 log.md。
- 高風險決策頁面用 status: needs_review，人工確認後才改 verified。

14.3 每週維護流程

1. 新增 raw_sources 文件。
2. 匯入 LightRAG，確認 processed。
3. 針對新文件建立 3-5 個 synthesis/query Wiki 頁。
4. 執行 scripts/wiki_lint.py。
5. 人工檢查 needs_review 頁面。
6. Git commit wiki/schema/scripts。
7. 備份 lightrag_workspace。

15. 常見錯誤排除

症狀	可能原因	處理方式
LightRAG 啟動失敗	.env 路徑或 API key 錯誤	先用最小 .env 啟動；檢查 lightrag-server log
Ollama embedding 連不上	ollama serve 未啟動或 port 不對	curl http://localhost:11434/api/tags 檢查
查詢結果很空	文件未 processed、query mode 不合、chunk 設定不合	先用 WebUI 查文件狀態，再用 mix 模式測試
Wiki 頁看起來漂亮但不可靠	沒有保存 references/chunk content	compile_wiki_page 必須 include_references + include_chunk_content
更換 embedding 後出錯	向量維度不一致	清除相關索引並重建；正式環境先備份
MiniMax API 回傳格式錯	endpoint/model/相容 API 差異	用 curl 先測 /v1/chat/completions，再套入 LightRAG
Agent 建立重複頁面	沒有先搜尋 Wiki	AGENTS.md 明確要求先查 index 與既有頁面
PDF 圖表內容漏掉	第一版 parser 沒開 multimodal/OCR	後續導入 MinerU/Docling/VLM，或先人工轉 Markdown

v1.1 新增錯誤排除項目

症狀	可能原因	處理方式
啟動時 WORKING_DIR 指向不存在路徑	.env 用單引號 heredoc 建立，\$USER 未展開成帳號名	把 WORKING_DIR 改成寫死的絕對路徑（見第 6 章 v1.1 警告）
server 啟動直接 fail fast	.env 殘留已淘汰的 ENTITY_TYPES 變數	移除 ENTITY_TYPES，改用 prompt profiles 機制
查詢時新舊版本內容混在一起	更新文件直接上傳新版，舊版索引未刪	先在 WebUI Documents 刪除舊版文件，再上傳新版（見 7.5）
重複上傳同一檔無任何警告	部分版本 WebUI 去重提示不明顯	上傳前到 Documents 列表確認狀態；以內容 hash 去重為準
編譯出的 Wiki 頁 Sources 全是 unknown	references 的 JSON 欄位結構與腳本假設不符	用 curl 實測 /query 回傳結構後調整 refs_to_markdown（見第 9 章補充）

16. 參考來源

以下來源用於確認截至 2026-06-11 的工具定位與設定方向。實際環境變數與 API 名稱仍應以各專案最新 README / env.example / Docs 為準。

- HKUDS LightRAG GitHub README：安裝 LightRAG Server、uv tool install、Docker Compose、query modes、embedding 建議、storage 類型。https://github.com/HKUDS/LightRAG
- HKUDS LightRAG API Server 文檔：WebUI/API、/query references、API key、parser pipeline、env examples。https://github.com/HKUDS/LightRAG/blob/main/docs/LightRAG-API-Server.md
- nashsu/llm_wiki GitHub README：LLM Wiki 桌面應用、Raw Sources → Wiki → Schema、MCP/API/Agent skill、Obsidian-compatible wiki。https://github.com/nashsu/llm_wiki
- nashsu/llm_wiki llm-wiki.md：Karpathy-style LLM Wiki pattern、index.md、log.md、wikilink、schema、lint/maintenance 思想。https://github.com/nashsu/llm_wiki/blob/main/llm-wiki.md
- Retrieval as Reasoning: Self-Evolving Agent-Native Retrieval via LLM-Wiki, arXiv:2605.25480。https://arxiv.org/abs/2605.25480
- MiniMax 官方平台與 OpenAI-compatible API 資訊應以平台控制台與官方文件為準。https://platform.minimaxi.com/

v1.1 補充：來源驗證提醒

v1.1 查證時已確認 LightRAG 官方倉庫的安裝方式、/query 的 include_chunk_content 參數、storage 後端選項與內容 hash 去重機制與本手冊敘述一致。**nashsu/llm_wiki 倉庫與 arXiv:2605.25480 論文連結未能獨立驗證**，依賴其中的 pattern 前請先實際開啟連結確認存在與內容。

17. 進階缺口與補強建議（v1.2 新增）

本章是系統實際運行前後值得補強的八個主題，按優先順序排列。前三項影響資料正確性與成本，建議在開始大量匯入文件前先讀。

v1.2 | 高優先

17.1 文件刪除後，知識圖譜不一定乾淨

LightRAG 匯入時會把不同文件中**同名的實體合併**成圖譜中的同一個節點。因此刪除某份文件時，被多份文件共同支撐的實體描述未必能完全回復到「彷彿這份文件從未存在」的狀態；社群也曾回報多份文件插入時節點與邊的合併行為不一致。

實務建議：

- 刪除文件後，針對該文件的核心主題跑幾個測試查詢，確認舊內容沒有殘留。
- 若殘留嚴重，考慮重建整個索引——這也是 raw_sources 必須完整保留的另一個理由。
- 重建前先閱讀 17.2 的 LLM cache 遷移，可大幅降低重建費用。

v1.2 | 高優先

17.2 匯入成本估算與煞車機制

實體抽取是「每個 chunk 至少呼叫一次 LLM」。一份 100 頁的 PDF 可能切出上百個 chunk，第一次匯入就是上百次 MiniMax API 呼叫。本手冊先前版本未提供任何成本估算指引。

實務建議：

- 第一份大文件匯入前，先用一份 5-10 頁的小文件實測，從 MiniMax 控制台看實際扣了多少 token，換算出「每頁成本」，再決定批量匯入的節奏。
- LightRAG 內建 LLM cache：相同內容重新處理時會命中快取大幅加速。官方文件指出切換儲存後端時可用遷移工具把既有 cache 搬到新儲存，重新上傳檔案時可省下重新抽取的大部分費用。將來重建索引（升 PostgreSQL、換 embedding）前，務必先了解並使用 cache 遷移工具（參見官方 README_MIGRATE_LLM_CACHE.md）。
- MAX_ASYNC_LLM 與 MAX_PARALLEL_INSERT 維持保守值（4 / 1），除了保護 16GB 記憶體，也避免短時間內大量 API 呼叫觸發費用爆衝。

v1.2 | 高優先

17.3 設計決策：Wiki 的知識要不要回流到 LightRAG？

目前的資料流是單向的：raw_sources → LightRAG → Wiki。Wiki 裡人工審核過的高品質結論（synthesis、comparison 頁）**LightRAG 並不知道**。當 Agent 直接查 LightRAG 時，得到的永遠只是原始文件的視角，看不到你已經整理好的判斷。

把 wiki/ 也匯入 LightRAG	維持單向（現狀）
檢索能命中整理後的知識，跨文件結論可被語意搜尋	架構簡單，職責清楚：LightRAG 只管原文證據
Wiki 改版後要同步更新索引，否則檢索到過期結論	Agent 必須遵守「先查 Wiki」的路由紀律才能用到整理後知識
可能造成「Wiki 引用 Wiki」的循環引用	—

建議：第一版維持單向即可，但把這個決策明確記在 schema/AGENTS.md 或一頁 concepts/ 決策頁中（含理由與日期），符合本手冊「凡是決策都要留下理由與日期」的原則。等 Wiki 累積到數十頁 verified 內容、且 Agent 路由紀律驗證可靠後，再評估是否開第二個 LightRAG workspace 專門索引 wiki。

v1.2 | 中優先

17.4 服務常駐化

手冊的啟動方式是前景執行 ollama serve 與 lightrag-server，終端機一關服務就停。長期使用需把兩個服務常駐化，設定開機自啟與崩潰自動重啟。

```
# Ubuntu systemd 範例：/etc/systemd/system/lightrag.service
[Unit]
Description=LighTRAG Server
After=network.target ollama.service

[Service]
User=你的帳號
WorkingDirectory=/home/你的帳號/hybrid-knowledge-lab
ExecStart=/home/你的帳號/.local/bin/lightrag-server
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target

# 啟用
sudo systemctl daemon-reload
sudo systemctl enable --now lightrag
```

macOS 改用 launchd plist (~/Library/LaunchAgents/)，Ollama 的 macOS 安裝版本身已含常駐 app。注意 systemd 模式下 .env 由 WorkingDirectory 讀取，正好呼應第 5 章 v1.1 補充的「.env 從工作目錄讀取」。

v1.2 | 中優先

17.5 文件卡在 processing 狀態的處理 SOP

社群曾回報大量文件卡在等待處理狀態、Scan 無效、必須手動停止並刪除卡住的文件才能讓佇列繼續。常見起因：OCR 失敗產生空白內容、特殊格式 PDF、LLM 呼叫逾時。

SOP：

1. 發現文件長時間（超過合理處理時間數倍）停在 processing → 查 lightrag-server log 找出卡住原因。
2. 在 WebUI Documents 停止並刪除該文件。
3. 依原因修正：空白/掃描型 PDF 先人工轉 Markdown 或留待 Multimodal 升級；逾時則調高 TIMEOUT 或降低 MAX_PARALLEL_INSERT。
4. 重新上傳，確認 processed 後在 wiki/log.md 記一筆事故與處理方式。

v1.2 | 中優先

17.6 備份要做還原演練

14.1 節的 tar.gz 備份從未驗證過「真的還原得回來」——備份沒測試過還原等於沒備份。建議在升級 PostgreSQL 之前至少做一次完整演練：

```
# 1. 解開備份到全新目錄
mkdir -p ~/restore-test && tar -xzf outputs/lightrag_workspace_YYYYMMDD.tar.gz -C ~/restore-test
# 2. 用臨時 .env 指向還原目錄，換一個 port 啟動
WORKING_DIR=~/restore-test/lightrag_workspace PORT=9622 lightrag-server
# 3. 跑一個已知答案的查詢，確認索引完好
curl -X POST http://localhost:9622/query -H "Content-Type: application/json" \
-d '{"query": "已知答案的測試問題", "mode": "mix", "include_references": true}'
```

wiki/ 的還原演練更簡單：git clone 到新目錄、用 Obsidian 打開確認連結完好即可。

v1.2 | 低優先

17.7 Git 衛生：outputs/ 與大型檔案

備份 tar.gz 放在 outputs/，但第 4 章的 .gitignore 沒有排除它。哪天 Agent 執行 git add，幾百 MB 的備份檔就進版本庫了。一行解決：

```
echo "outputs/" >> .gitignore
```


另外 raw_sources 放大量 PDF 會讓 Git repo 持續膨脹（Git 對二進位檔的差異儲存效率差）。文件多了之後考慮 git-lfs 管理 raw_sources 中的 PDF，或將 raw_sources 改為獨立的備份策略（rsync/雲端同步）而非 Git。

v1.2 | 低優先

17.8 建立檢索品質的黃金問題集

附錄 A 驗收的是「系統會動」，不是「答案是對的」。建議建一組 5-10 題的黃金問題集：你自己知道標準答案、且答案明確存在於某份文件中的問題。

- 每次大改動後（換 chunk 設定、開 rerank、升級 LightRAG 版本、換模型）重跑一遍，比對答案品質有沒有退步。
- 這組問題正好可以放在 wiki/queries/ 當第一批頁面：問題、標準答案、預期引用來源都寫進頁面，回歸測試時逐頁比對。
- 進一步可把比對自動化成一個 scripts/golden_check.py，逐題呼叫 /query 並檢查回應是否包含關鍵字與正確的 reference file_path。

附錄 A：最小可行驗收清單

項目	驗收方式	通過標準
LightRAG Server	curl /health	回傳正常狀態
Embedding	ollama list	看到 bge-m3
文件匯入	WebUI document status	processed
查詢	/query include_references	有 response + references
Wiki 編譯	compile_wiki_page.py	wiki/queries 產生 md 頁面
Wiki Lint	wiki_lint.py	沒有 missing Sources 的 verified 頁
Agent 使用	提示詞測試	先查 Wiki，不足時查 LightRAG
備份	git commit + tar.gz	可還原 wiki 與 workspace

v1.1 新增驗收項目

項目	驗收方式	通過標準
去重機制	重複上傳同一份檔案	不重跑 LLM 抽取，Documents 不出現重複處理
references 結構	curl /query 實測並檢視 JSON	compile 腳本解析出正確 file_path 與摘錄
.env 路徑	cat .env 檢查 WORKING_DIR	為展開後的絕對路徑，無字面 \$USER

附錄 B：建議落地順序

- 第一天：裝 Ollama、bge-m3、LightRAG Server，確認 /health。
- 第二天：匯入 3-5 份 Markdown 或 README，測試 /query references。
- 第三天：建立 wiki/index.md、log.md、AGENTS.md。
- 第四天：跑 compile_wiki_page.py 產生第一批 Wiki 頁。
- 第五天：讓 Claude Code/Codex/Hermes 用 AGENTS.md 操作 Wiki。
- 第六天：加入 wiki_lint.py，建立 needs_review 流程。
- 第七天：決定是否升 PostgreSQL、Rerank、MCP Server。

v1.1 補充：第七天之後

流程穩定後，可依序考慮：(1) 切換到 INPUT_DIR + /documents/scan 自動掃描模式（見 7.6，建議搭配 inbox/ 待匯入區）；(2) 把 scripts/ 包成 MCP tools（見 12.2）；(3) 升級 PostgreSQL 前先完成一次完整備份演練。